RAPPORT DE STAGE

Nom : BLANIC Clément

Formation : BTS SIO1

Entreprise : ENEDIS

Adresse : 32 Avenue de l'Europe 92400 Courbevoie

Stage du : 26 mai 2025 au 6 juillet 2025

Maître de stage : ILCINKAS Chrystelle

Tuteur de stage : BENYOUSSEF Redouane

Sommaire

- **Introduction**
  Présentation du sujet de stage chez Enedis, dans une équipe de développement autour des compteurs Linky, et annonce du plan du rapport.

- **Chapitre 1 : Présentation de l'entreprise**
  Description d'Enedis, du service d'accueil et du contexte technique (équipements, architecture, outils).

- **Chapitre 2 : Cahier des Charges**
  Définition des objectifs liés à l'automatisation des tâches, présentation des attentes du tuteur et des outils utilisés.

- **Chapitre 3 : Travail accompli**
  Détail des scénarios automatisés (comme SMC14 et SMC32), explication des étapes, fonctions utilisées, difficultés rencontrées et résolutions.

- **Conclusion**
  Bilan du travail réalisé, compétences techniques acquises, découverte du travail en équipe et retour personnel sur l'expérience.

- **Annexes**
  - Annexe 1 : Schéma du système Informatique
  - Annexe 2 : Exemple résultat d'un scenario sur Chutney

Introduction

As part of my academic preparation, I finished a technical internship at Enedis, a significant player in the French electrical distribution system. In order to create automation tools for the Linky smart meter ecosystem, I joined a cross-functional development team.

The goal of my internship was to **automate repetitive tasks** described in user support tickets. These tasks often involved a series of actions on web interfaces, such as launching services according to tasks needed, specific buttons, navigating through menus, or verifying databases and logs. The aim was to create automated scripts that could carry out these tasks with minimal user intervention. Among the tools and technologies, I worked with were **Kotlin**, **Chutney**, **DBeaver**, and **IntelliJ IDEA**.

This report is structured as follows:

- The **first chapter** presents the **business context** and the **technical environment** in which I worked.
- The **second chapter** outlines the **objectives of the internship**, based on the department's needs and my supervisor's expectations.
- The **third chapter** details the **work accomplished**, including the development of automation scenarios, the tools used, and the challenges encountered during implementation.
- Finally, I reflect on the **outcomes of this experience**, both technically and professionally.

This internship allowed me to deepen my programming skills in a real-world context and to better understand the industrial challenges of large-scale automation in the energy sector.

Chapitre 1 : Présentation de l'entreprise

Enedis, fondée en 2008, dans le contexte de l'ouverture du marché de l'électricité à la concurrence, est une entreprise de distribution d'électricité et gestionnaire du réseau électrique, spécialisée dans l'exploitation et la modernisation du réseau électrique, l'accompagnement des projets et collectivités locales. Elle assure l'équilibre du système électrique, propose un dépannage 24/24H, 7/7j et l'intervention clientèle, la gestion du comptage des consommations pour les fournisseurs d'énergies et enfin le raccordement aux réseaux des clients.

J'ai pu faire mon stage dans le Pôle Nex'us (Systèmes et objets communicants), dans l'équipe de recette transverse qui réalise les tests de validation et de cohérence de bout en bout sur le système Linky avec l'ensemble des partenaires

Annexe 1 : Schéma du système Informatique

Lors de ce stage, j'ai intégré une équipe chargée de développer des programmes permettant d'automatiser certaines actions liées au compteur Linky, comme la modification de la puissance d'un concentrateur, et autres actions.

Technologies, outils et langages utilisés :

- **Langage :** Kotlin
- **Automatisation :** Chutney
- **Formats de données :** JSON, XML
- **Environnement de développement :** IntelliJ IDEA
- **Base de données :** DBeaver
- **Sécurité / Connexion :** FortiClient VPN, OpenTrust CMS (clé USB)
- **Applications internes :** LinkySup, Idatha Log

Durant ce stage j'ai pu utiliser différents outils de développement :

Kotlin est un langage de programmation open source qui combine la programmation orientée objet et la programmation fonctionnelle. Il compile pour la machine virtuelle Java (JVM) ainsi que pour JavaScript. Dans notre cas, nous avons utilisé **IntelliJ IDEA**, un environnement de développement intégré (IDE) développé par JetBrains, particulièrement adapté pour Kotlin.

Chutney est un outil d'automatisation open source développé par Enedis. Il permet d'enchaîner des scénarios complexes de manière simplifiée. Grâce à Chutney, nous pouvons suivre en détail l'exécution des tests, y compris les éventuelles erreurs, ce qui facilite grandement le débogage et la validation des scénarios automatisés.

Annexe 2 : Exemple résultat d'un scenario sur Chutney

DBeaver est un outil de gestion de bases de données, utilisé pour interagir avec les données via le langage SQL. Il permet de consulter, modifier ou interroger facilement les bases de données.

IdathaLog est une plateforme web interne à l'entreprise, accessible uniquement via une connexion sécurisée grâce à un VPN (FortiClient) et une clé USB PKI (OpenTrust CMS). Ce site permet de consulter les différents logs générés lors de l'exécution des scénarios automatisés avec Chutney.

LinkySup est un logiciel interne utilisé pour détecter et diagnostiquer les dysfonctionnements du système. Il permet également d'administrer le système et de traiter les anomalies détectées.

Bitbucket est un outil de gestion de code basé sur Git, similaire à GitHub. Il est utilisé en équipe pour gérer les versions du code, faciliter la collaboration et intégrer de nouvelles fonctionnalités dans les projets.

D'autres formats et langages comme **XML** et **JSON** sont également utilisés. Ils sont souvent générés automatiquement par IntelliJ IDEA et Chutney pour permettre le bon fonctionnement des programmes et des scénarios d'automatisation.

Dans les locaux où j'ai réalisé mon stage, j'étais installé à proximité des **salles de tests**, spécialement aménagées pour effectuer des essais sur les équipements liés au réseau électrique intelligent.

<u>Images de la salle des tests :</u>

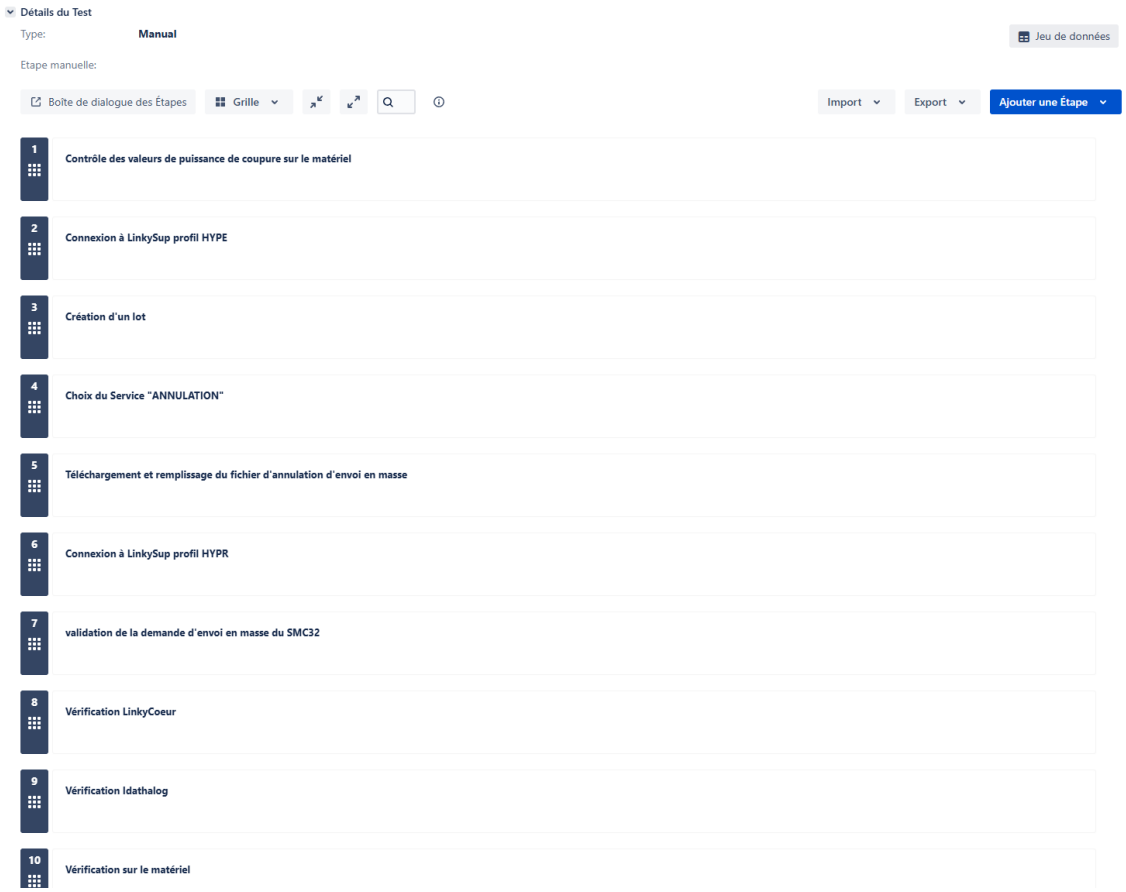
Ces salles permettent de **simuler et reproduire des scénarios réels** dans un environnement maîtrisé, afin de tester le comportement des équipements, valider les développements ou diagnostiquer des dysfonctionnements.

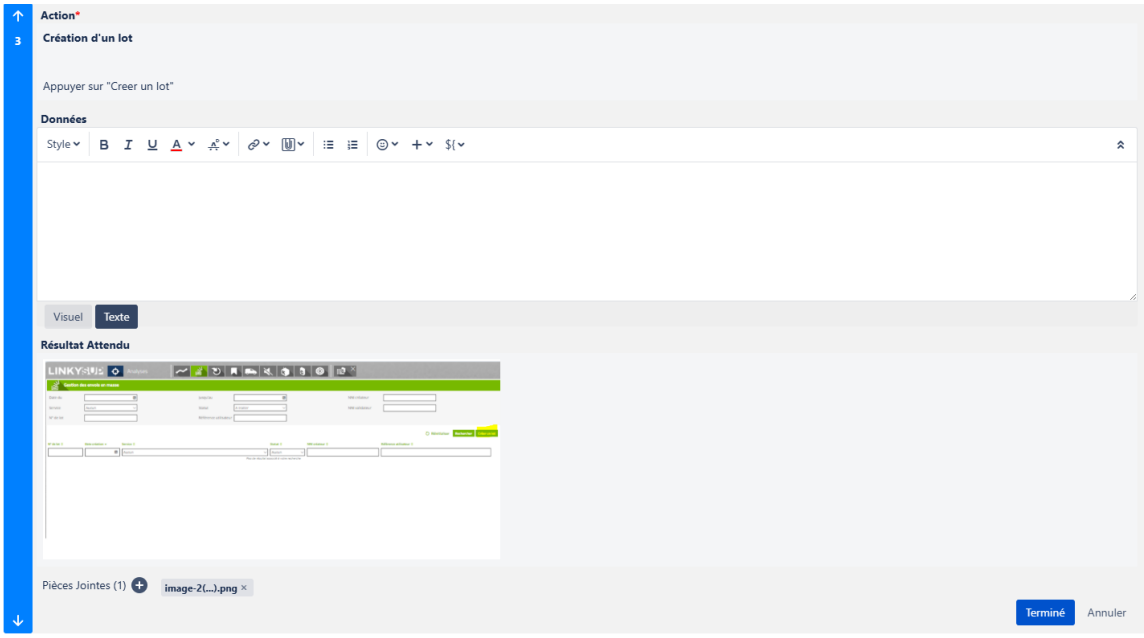
Chapitre 2 : Cahier des Charges

Mes missions consistaient principalement à analyser en détail les tickets reçus, puis à développer des scripts permettant d'automatiser les actions demandées. L'objectif était que l'utilisateur n'ait plus qu'à exécuter le programme, et que l'ensemble des actions décrites dans le ticket soient réalisées automatiquement, sans intervention manuelle.

Les tickets sont généralement très précis : ils détaillent, étape par étape, les actions à effectuer. Cela peut inclure, par exemple, le clic sur un onglet spécifique d'un site, la navigation dans une interface ou encore la saisie de données dans un champ particulier.

Exemple de ticket (illustration visuelle) :





Un ticket est souvent structuré en plusieurs **étapes**, chacune correspondant à une **action bien définie**. Lorsqu'on consulte une étape plus en détail, on y retrouve généralement une description textuelle de l'action à effectuer, souvent accompagnée d'une **image** pour faciliter la compréhension.

Rôle dans le processus

Mon rôle était de concevoir et de programmer un scénario automatisé capable d'enchaîner ces actions dans l'ordre indiqué. Une fois le script terminé et validé, il pouvait être utilisé quotidiennement pour exécuter ces tâches de manière automatique, fiable et répétable, sans qu'un utilisateur ait besoin de les faire manuellement.

Types de Scénarios :

WS (Web service) : Appelle directement le serveur en http sans passer par le navigateur

IHM (Interface Homme Machine) : Scénario qui passe par le navigateur et qui simule les actions d'un utilisateur

SMC (Service Métier Comptage) : Scénario qui répond à une demande du SI (Service Informatique), en effectuant 1 ou plusieurs traitements dans le SI Linky Coeur

SIL (Service Interne Linky) : C'est un service uniquement interne à Linky

PIL (Processus Interne Linky) : Processus déclenché lors d'un levier d'évènement déclencheur

Chapitre 3 : Travail accompli

Les taches que j'ai pu effectuer :

SMC14 coté IHM, « FOURNITURE DE CERTIFICATS MAINTENEUR »

```
@Scenario  1 Usage  Redouane BENYOUSSEF
fun smc14() = withData(
    ...elements = Data(
        id = 320,
        title = "[SMC14][LSUP] FOURNITURE DE CERTIFICATS MAINTENEUR",
        description = "",
        //dataset = PIL16_DATASET.createDefault(),
    )
) {
    //val idK = (it.dataset as PIL16_DATASET).K_ID

    Scenario(id = it.id, title = it.title) {
        Given( description = "Etant donnée un utilisateur connecté à LINKYSUP") {
            goToLinkySup()
            loginLinkySup( user = LINKYSUP_USER.HYPERVISEUR_R)
        }
        And( description = "Et depuis l'ecran analyse linkycoeur") {
            click( uiItem = Repository().mainPage.WORKSPACE_BTN,  description = "Ouvrir menu workspace")
            click( uiItem = Repository().mainPage.workspace_menu.linkycoeur_btn,  description = "Sélectionner le menu linkycoeur")
            linkysupWaitLoading()
        }
        When( description = "Récupère le certificat"){
            sendKeys( uiItem = Repository().linkyCoeurPage.GENERER_CERTIFICAT_PWD_INPUT, value = "password_certificat_SMC14".spEL, description = "Saisir mot de passe")
            click( uiItem = Repository().linkyCoeurPage.RECUPERER_CERTIFICATS_BTN, description = "Récupère le certificat", outputs = dateReceptionToMap)

        }
        Then( description = "La demande est traitée avec succés dans BD") {
            validationRDS(idService = "SMC14")
        }

        And( description = "Et pas d'erreur/warning dans idatha logs") {
            val idDemande = "firstRow.get('ID_DEMANDE')".spEL()
            verificationLogLinkycoeur(
                query = "$idDemande OR SMC14"
            )
        }
```

Exemple de scénario automatisé – Structure et fonctionnement

Sur l'image ci-dessus, on peut observer la structure de base d'un scénario automatisé. Cette partie du code permet de chaîner plusieurs actions à l'aide de balises spécifiques telles que **Given**, **When**, **Then** et **And**. Ces balises, associées à des descriptions, facilitent la compréhension du scénario et permettent de mieux s'orienter dans le code, qu'il soit en **Kotlin**, **XML** ou **JSON**.

Chaque balise contient une ou plusieurs **fonctions**. Certaines de ces fonctions, comme vérificationLogLinkyCoeur, sont développées en interne dans d'autres fichiers du projet. D'autres, comme click, proviennent du langage utilisé ou d'un **framework**, et intègrent des **paramètres prédéfinis** stockés dans des fichiers séparés.

L'utilisation de ces fonctions prédéfinies rend le code plus modulaire et maintenable. En effet, si une modification survient sur l'interface (comme celle du site LinkySup), il suffira de mettre à jour les paramètres dans le fichier centralisé, sans avoir à modifier tous les scénarios un par un.

```
18  fun LinkySupUiItem.getSelector(): String {  12 Usages  Redouane BENYOUSSEF
19      var selector = this.selector
20      if (this.by == SELENIUM_BY.xpath && this.selector.contains( other = "ZK:")) {
21          selector = selector.zkResolve
22      }
23      repositoryParams.forEach { selector = selector.replace( oldValue = "**" + it.key + "**",  newValue = it.value) }
24      return selector
25  }
26
27  fun ChutneyStepBuilder.click(  Redouane BENYOUSSEF
28      uiItem: LinkySupUiItem,
29      description: String,
30      strategy: Strategy? = null,
31      outputs: Map<String, Any> = mapOf()
32  ) = Step(description, strategy) {
33      click(uiItem, outputs)
34  }
```

Comme bon exemple de fonction prédéfinie utilisée dans nos scénarios il y a la fonction **click**, définie dans un fichier externe. Cette fonction permet de simuler un **clic automatisé** sur un élément d'une application web, quel qu'il soit.

En regardant de plus près son implémentation, on observe qu'elle repose sur **Selenium**, un **framework open source** largement utilisé pour automatiser les interactions avec des interfaces web (clics, remplissage de champs, navigation, etc.).

```
click( uiItem = Repository().mainPage.WORKSPACE_BTN,  description = "Ouvrir menu workspace")
```

L'utilisation de cette fonction est simple : il suffit de lui indiquer l'élément sur lequel cliquer. Dans notre cas, il s'agit du **bouton "WORKSPACE"**. Les **paramètres nécessaires pour localiser cet élément** sont définis dans un fichier séparé, ici nommé mainPage.

```
val WORKSPACE_BTN = LinkySupUiItem( label = "WORKSPACE_BTN",  by = zk,  selector = "currentWorkspaceIcon")
```

Le bouton est représenté par une **variable**, et le **selector** correspond à la clé (ou sélecteur CSS/XPath) identifiée en inspectant le site **LinkySup** via l'outil de développement du navigateur. Cela permet à Selenium de cibler précisément l'élément à cliquer.

Exemple concret : Scénario SMC14 (côté IHM)

Le scénario **SMC14**, par exemple, a pour objectif de **générer un certificat** via l'interface IHM et de le **récupérer**. Pour cela, le script va :

1. Connexion à l'interface

   Le script accède au site et saisit les identifiants nécessaires. Dans notre cas, nous utilisons le compte **HYPERVISEUR_R**.

2. Naviguer dans les menus de l'application

   La navigation dans l'interface se fait grâce à une succession de **clics automatisés** permettant d'accéder au menu **LinkyCoeur**.

3. Lancement la génération du certificat et de la récupération du certificat

   À l'aide de la commande **sendKeys**, le script renseigne les champs de texte requis pour déclencher la génération du certificat.

4. Vérifications

   Enfin, effectuer des vérifications dans la **base de données** via **DBeaver** et consulter les **logs sur IdathaLog** pour s'assurer que la demande s'est bien déroulée et qu'aucune erreur n'a été rencontrée.

Difficultés rencontrées

L'une des principales difficultés rencontrées durant le développement concernait l'exécution d'un scénario dans lequel le bouton **« Récupérer le certificat »** apparaissait grisé. En conséquence, le programme poursuivait son exécution sans effectuer l'action attendue, ce qui faussait les résultats.

Après investigation, il s'est avéré que le problème venait des **droits d'accès associés à l'identifiant utilisé**. En effet, certaines actions sur l'interface nécessitent des permissions spécifiques. J'ai donc sollicité l'aide d'un collègue pour identifier **les identifiants corrects** permettant d'avoir les droits suffisants. Une fois ces informations obtenues, j'ai pu modifier les paramètres d'authentification dans la fonction loginLinkySup () afin que le script fonctionne correctement.

SMC32 coté IHM, « Ecrêtement Ciblé – Annulation »

```
24  @Scenario  1 Usage  CB4A4C2L
25  fun smc32() = withData(
26      ...elements = Data(
27          id = 321,
28          title = "[SMC32][LSUP]Ecrêtement Ciblé - Annulation",
29          description = "",
30          dataset = SMC32_01_DATASET.createDefault(),
31      )
32  ) {
33
34
35      Scenario(id = it.id, title = it.title) {
36          val dataset = it.dataset as SMC32_01_DATASET
37          Given( description = "Etant donnée un utilisateur connecté à LINKYSUP") {
38              goToLinkySup()
39              loginLinkySup( user = LINKYSUP_USER.HYPERVISEUR_E)
40          }
41          And( description = "Accèder à l'onglet Gestion des envois en masse") {
42              linkysupSelectionOngletPageRechercheAction(onglet = "GESTION")
43          }
44          When( description = "Demande d'annulation d'ecrêtement ciblé (SMC32_01)") {
45              linkysupEnvoiEnMasseAction(
46                  service = "SMC32_01",
47                  fileName = "envoi_masse_SMC32_01_Annulation",
48                  fileContent = """
49                      prm
50                      ${"#jsonPath(#dataset,'$..PRM_ID').toString().replaceAll('[\\[\\]\\\*\\s]+','').replaceAll(',',T(java.lang.System).lineSeparator())".elEval()}
51                  """.trimIndent(),
52                  delay = "1",
53                  outputs = dateReceptionToMap
54              )
55          }
56          And( description = "Se connecter a linkysup avec profil HYPR afin de valider la demande") {
57              logoutLinkySup()
58              loginLinkySup( user = LINKYSUP_USER.HYPERVISEUR_R)
59          }
60          And( description = "Accèder à l'onglet Gestion des envois en masse") {
61              linkysupSelectionOngletPageRechercheAction(onglet = "GESTION")
```

```
63          And( description = "Validation de la demande SMC32_01") {
64              ValidationDemandeEnvoiEnMasse(service = "SMC32_01")
65          }
66          Then( description = "La demande est traitée avec succés dans BD", strategy = ForStrategy()) {
67              validationRDS(idService = "SMC32")
68          }
69
70          And( description = "Et pas d'erreur/warning dans idatha logs") {
71              val idDemande = "firstRow.get('ID_DEMANDE')".spEL()
72              verificationLogLinkycoeur(query = "$idDemande ")
73              verificationLogLinkysup(query = "$idDemande ")
74              verificationLogLinkycom(query = "${dataset.K_ID} AND SMC32")
75          }
76
77          And(
78              description = "Vérification du compte rendu de la demande",
79              strategy = retryWithTimeout( timeOut = "60 sec", retryDelay = "5 sec")
80          ) {
81              val idDemande = "firstRow.get('ID_DEMANDE')".spEL()
82              verificationLogLinkycom(
83                  query = "\\[$idDemande AND SMC32",
84                  levelMin = info,
85                  outputs = mapOf("message_<i>" to "#jsonPath(#body,\"$.logs[?(@.message =~ /.*\"+ #K_ID +\".*/)].message\")[0].split('\\] , \\[')[2].replace(']','')
86                  validations = mapOf("C_ID présent" to "#jsonPath(#body,\"$.logs[?(@.message =~ /.*\"+ #K_ID +\".*/)].message\")[0].split('\\] , \\[')[2].replace(']
87              )
88
89          }
90
91
92      }
93
94
95  }
96
97
```

Le scénario **SMC32**, a pour objectif de **faire des Ecrêtements ciblés** via l'interface IHM. Pour cela, le script va :

1. Connexion à l'interface

   Le script accède au site et saisit les identifiants nécessaires. Cette fois, il utilise le compte **HYPERVISEUR_E**, un compte avec moins de droits que **HYPERVISEUR_R**.

2. Naviguer dans les menus de l'application

   Comme pour le scénario SMC14, il se déplace dans les menus de l'application afin d'accéder à l'onglet **Gestion des envois de masse**.

3. Création de l'écrêtement ciblés

   Du coup à l'aide d'une fonction on va entrer plusieurs paramètres comme une ligne de code en JsonPath pour venir rendre la demande complète.

4. Déconnexion puis Reconnexion

   Le script se déconnecte, puis se reconnecte avec un compte **HYPERVISEUR_R** disposant de **droits supplémentaires**.

5. Valider la demande

   À l'aide d'une fonction qui regroupe plusieurs actions, le script valide la demande d'écrêtement.

6. Vérification de la BDD

   Grâce à l'Id Service, il vérifie si la demande a bien été prise en compte.

7. Vérification des logs

   Le script consulte les logs des différentes infrastructures (**LinkyCom**, **LinkyCoeur**, **LinkySup**) afin de détecter d'éventuelles erreurs via des requêtes utilisant des ID.

8. Vérification du compte rendu de la demande

   Enfin, il vérifie si le compte rendu est valide grâce à différents **ID** et aux messages analysés via du code en **JsonPath**.

Difficultés rencontrées

J'ai eu du mal à vérifier les données, car je ne savais pas bien utiliser JsonPath. Sur ce point, j'ai donc été beaucoup aider par mon tuteur. J'ai également rencontré des difficultés pour vérifier les logs, notamment pour extraire certaines données et ID depuis Chutney et les utiliser ensuite dans des requêtes de vérification**.**

De plus, selon le **dataset** utilisé, ce programme peut exécuter plusieurs écrêtements en une seule exécution. J'ai donc rencontré des difficultés pour **mettre en place cette stratégie d'itération**, car il fallait utiliser la **ForStrategy()** , que nous avons intégrée et cachée dans certaines fonctions, mais on peut la voir visible ici :

```
Then( description = "La demande est traitée avec succés dans BD", strategy = ForStrategy()) {
    validationRDS(idService = "SMC32")
}
```

Conclusion

Au terme de ce stage, j'ai pu tirer un bilan très positif de mon expérience

D'une part, mon travail a permis de **contribuer aux campagnes d'automatisation** mises en place par l'entreprise. J'ai conçu et développé **deux programmes,** qui ont vocation à être intégrés dans des processus quotidiens pour automatiser des tâches.

D'autre part, ce stage m'a offert une **riche expérience de développement**. J'ai pu progresser dans l'utilisation de différents outils et langages : **Kotlin, Chutney, SQL via DBeaver**, et j'ai appris à naviguer dans des environnements complexes comme des architectures informatiques industrielles et des outils internes tels que **LinkySup** ou **IdathaLog**. Ces compétences techniques ont été renforcées par la nécessité de comprendre les systèmes existants, de relire et réutiliser du code déjà écrit.

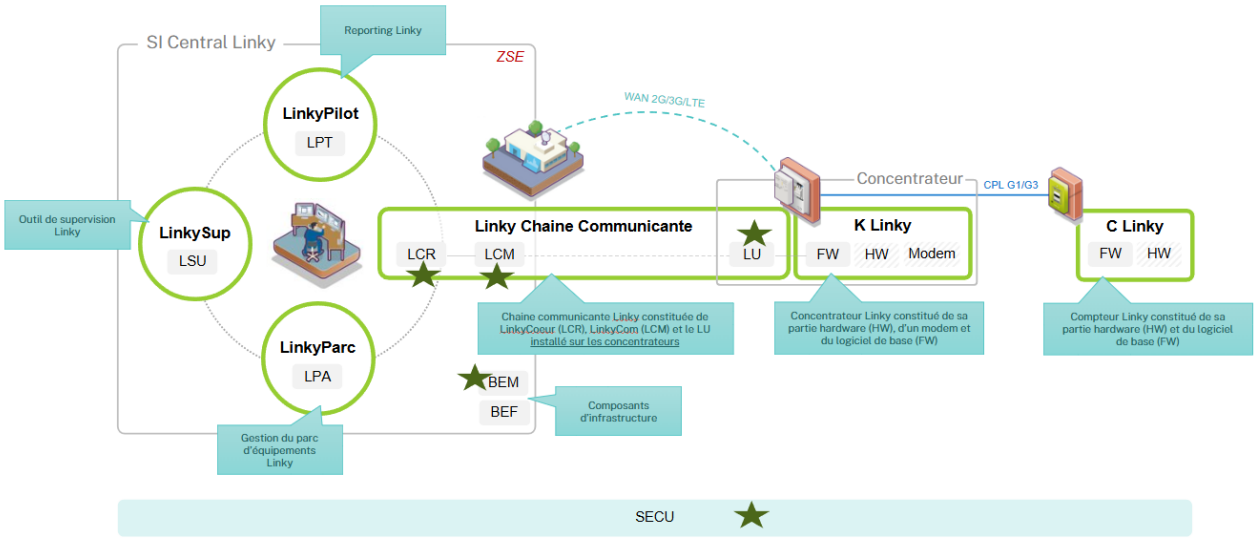
Par ailleurs, j'ai eu l'occasion de **me former à la cybersécurité en entreprise**, grâce à des formations spécifiques proposées en interne. Celles-ci m'ont sensibilisé à des aspects essentiels comme le respect de la **charte informatique**, les bonnes pratiques de sécurité (verrouiller sa session avant de quitter son poste, choisir des mots de passe robustes et non réutilisés…), et la protection des données.

Enfin, ce stage m'a permis de **m'intégrer à un environnement professionnel**, de comprendre les **modes de communication au sein d'une équipe**, et de mesurer la **complexité des projets menés en collaboration**. J'ai appris à interagir efficacement avec mes collègues, que ce soit en face à face ou à distance via des outils comme **Teams**, afin de résoudre des problèmes techniques, obtenir des informations. J'ai également découvert les enjeux de la coordination et du suivi dans un projet impliquant plusieurs intervenants, en assistant à plusieurs réunions.

En conclusion, cette expérience m'a beaucoup apporté sur les plans **technique**, **organisationnel** et **humain**. Elle m'a conforté dans mon envie de poursuivre dans le domaine du développement, et m'a donné un aperçu concret des défis rencontrés dans le monde professionnel.

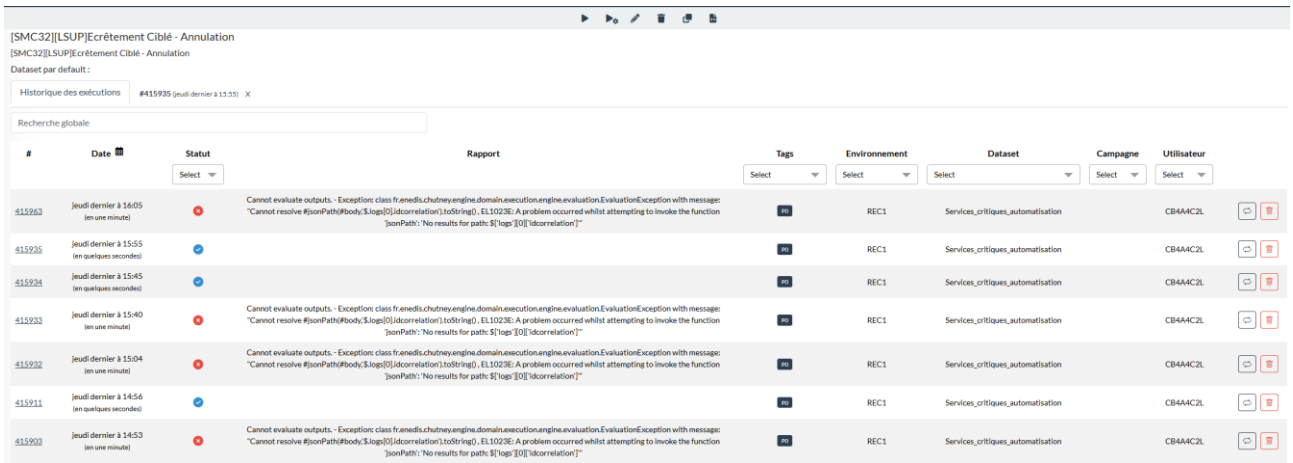
Annexes

Annexe 1 : Schéma du système Informatique



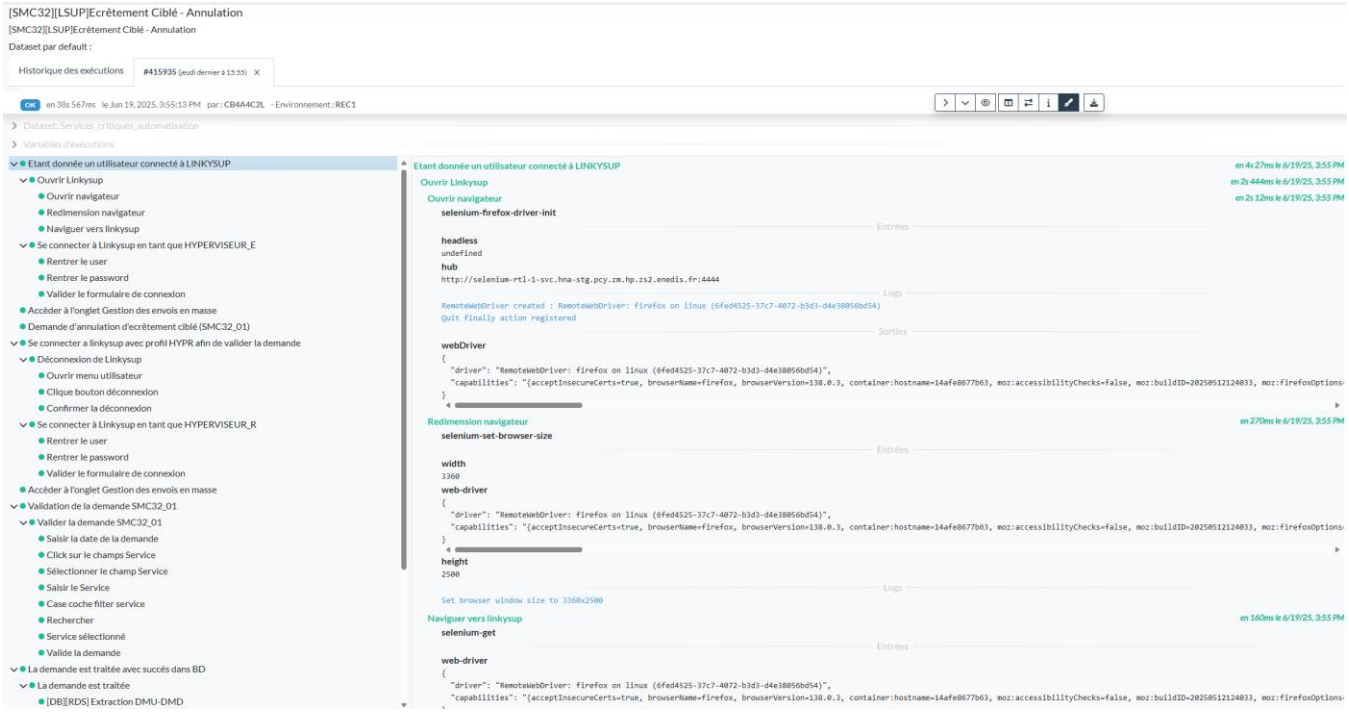
RETOUR A LA PAGE PRECEDENTE

Annexe 2 : Exemple résultat d'un scenario sur Chutney



Sur ce premier écran de **Chutney**, nous pouvons visualiser toutes les exécutions du scénario **SMC32**. Plusieurs informations y sont affichées pour chaque exécution :

- **l'ID** de l'exécution,
- **la date** et l'heure d'exécution,
- **le statut** : une croix rouge indique une erreur survenue pendant l'exécution, un rond bleu signifie que le script est valide et s'est déroulé correctement, et un rond gris montre que le script a été interrompu manuellement avant la fin,
- **les tags** qui permettent de trier ou filtrer les scripts selon différents critères,
- **l'environnement de simulation** utilisé pour l'exécution ; dans notre cas, il en existe trois : **REC1**, **REC2** et **REC3**,
- **la base de données** sur laquelle le scénario s'appuie,
- **la campagne** si le scénario fait partie d'un ensemble de tests automatisés,
- et enfin **l'utilisateur** qui a lancé le scénario.



Et sur ce second screen, on peut voir en détail le scénario avec toutes les étapes exécutées. Une pastille de couleur (rouge ou verte) indique si une erreur est survenue ou non. On retrouve aussi des détails comme des ID, des actions web et d'autres informations.

RETOUR A LA PAGE PRECEDENTE